

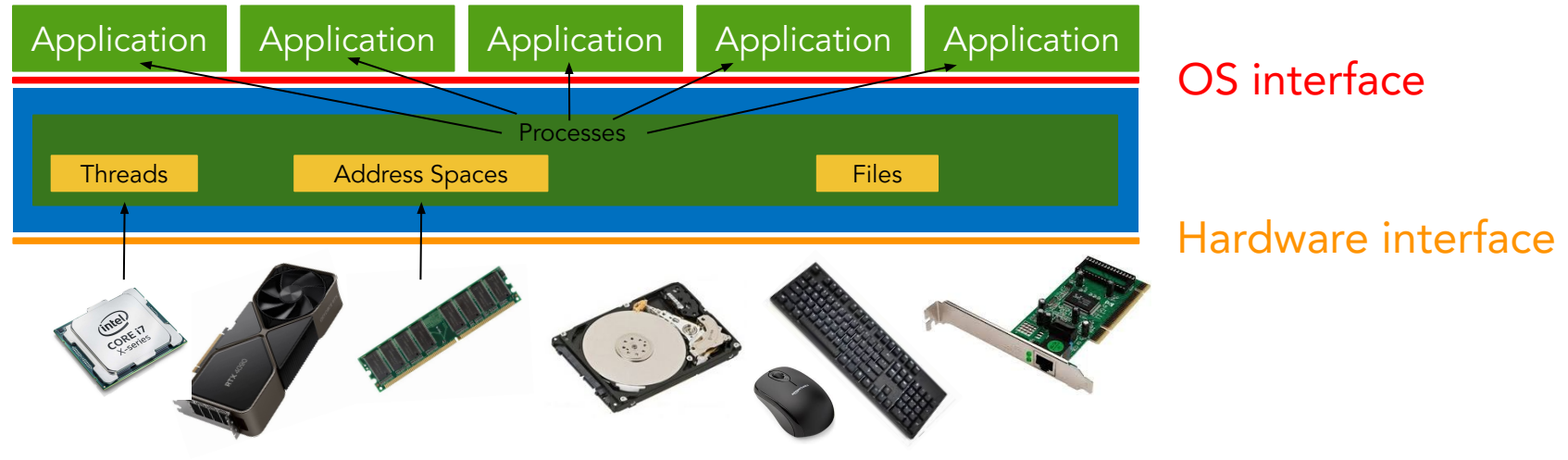
Lecture 9: File Systems

CS211 - Operating systems

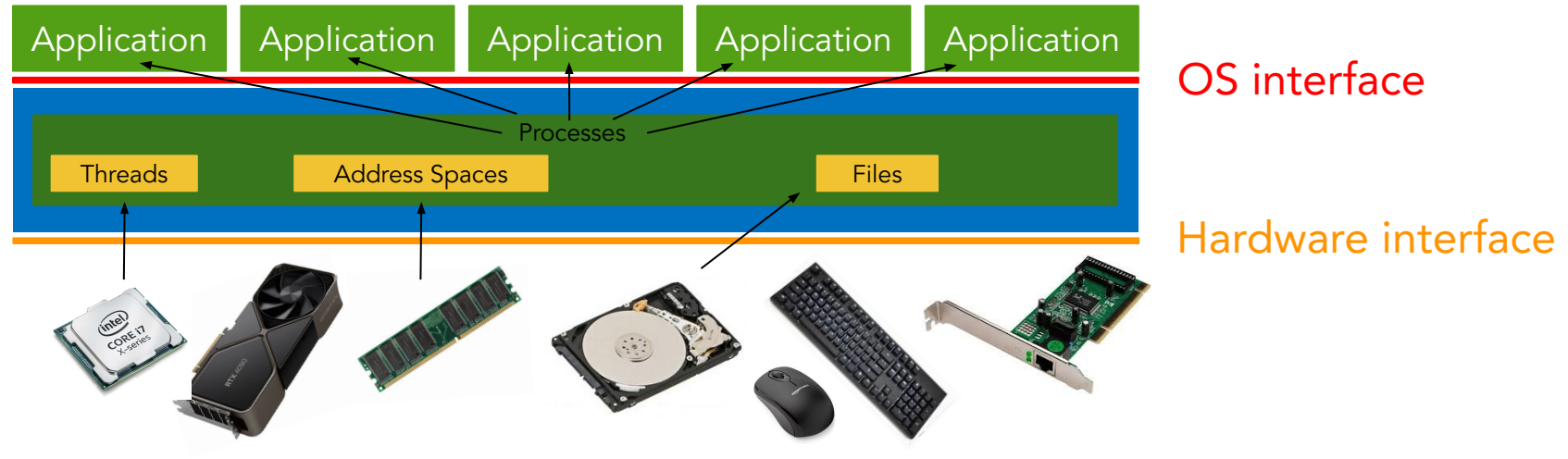
Spring 2026



Files



Files



Why file systems?

Why file systems?

- Where shall we store our data?
 - Process Memory?

Why file systems?

- Where shall we store our data?
 - Process Memory?
 - Bad idea!

Why file systems?

- Where shall we store our data?
 - Process Memory?
 - Bad idea!
- Need a way to store long-term information
 - Large amounts of information
 - Information must survive processes
 - Need concurrent access by multiple processes

Why file systems?

- Where shall we store our data?
 - Process Memory?
 - Bad idea!
- Need a way to store long-term information
 - Large amounts of information
 - Information must survive processes
 - Need concurrent access by multiple processes
- Solution

Why file systems?

- Where shall we store our data?
 - Process Memory?
 - Bad idea!
- Need a way to store long-term information
 - Large amounts of information
 - Information must survive processes
 - Need concurrent access by multiple processes
- Solution
 - A file system
 - Presents applications with named + persistent data
 - Two main components:
 - Files
 - Directories

File abstraction

File abstraction

- File = a named collection of data
 - has two parts

File abstraction

- File = a named collection of data
 - has two parts
 - data: what a user or application puts in it
 - array of untyped bytes
 - metadata: information added and managed by the OS
 - size, owner, security info, modification time, ...

File abstraction

- File = a named collection of data
 - has two parts
 - data: what a user or application puts in it
 - array of untyped bytes
 - metadata: information added and managed by the OS
 - size, owner, security info, modification time, ...
- File = abstracted unit of information

File abstraction

- File = a named collection of data
 - has two parts
 - data: what a user or application puts in it
 - array of untyped bytes
 - metadata: information added and managed by the OS
 - size, owner, security info, modification time, ...
- File = abstracted unit of information
 - Don't care exactly where on disk the file is
 - Files have human readable names
 - file given name upon creation
 - use the name to access the file

File name

File name

- Naming conventions

File name

➤ Naming conventions

- Some things OS dependent:
 - Windows not case sensitive (Windows 10), UNIX is
- Some things common:
 - Usually ok up to 255 characters

File name

- Naming conventions
 - Some things OS dependent:
 - Windows not case sensitive, UNIX is
 - Some things common:
 - Usually ok up to 255 characters
- File extensions, OS dependent:

File name

➤ Naming conventions

- Some things OS dependent:
 - Windows not case sensitive, UNIX is
- Some things common:
 - Usually ok up to 255 characters

➤ File extensions, OS dependent:

- Windows:
 - attaches meaning to extensions
 - associates applications to extensions
- UNIX:
 - extensions not enforced by OS
 - Some apps might insist upon them (.c, .h, .o, .s, for C compiler)

Directory

Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory

Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory

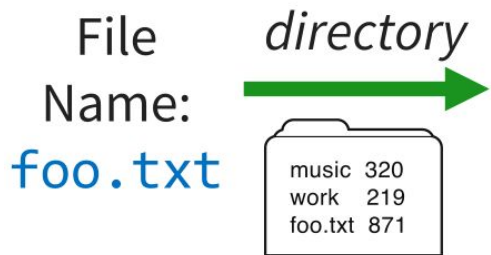
File

Name:

foo.txt

Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory



Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory



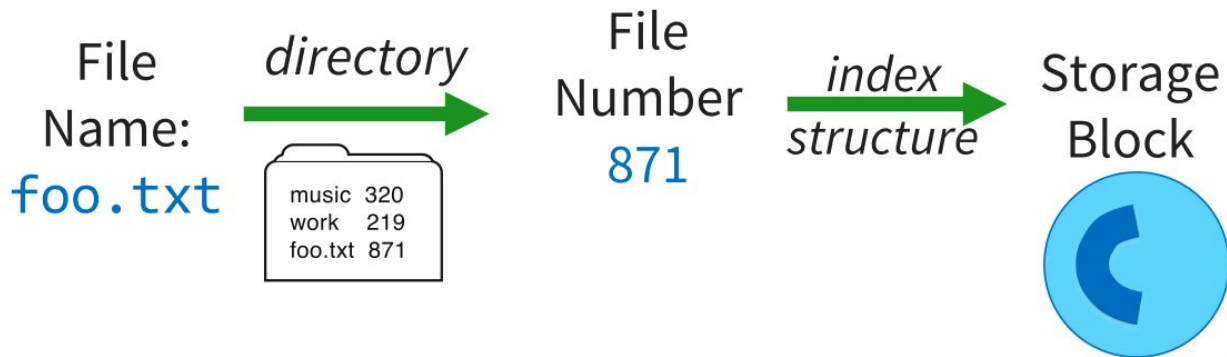
Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory



Directory

- Directory: provides names for files
 - a list of human readable names
 - a mapping from each name to a specific underlying file or directory



Path names

Path names

- Two types

Path names

- Two types
 - Absolute: path of file from the root directory
 - /home/ada/projects/babbage.txt
 - Relative: path from the working directory
 - projects/babbage.txt

Path names

➤ Two types

- Absolute: path of file from the root directory
 - `/home/ada/projects/babbage.txt`
- Relative: path from the working directory
 - `projects/babbage.txt`
 - How to know where is the current working directory?

Path names

➤ Two types

- Absolute: path of file from the root directory
 - /home/ada/projects/babbage.txt
- Relative: path from the working directory
 - projects/babbage.txt
 - How to know where is the current working directory?
 - PCB

OS uses path name to find directory

OS uses path name to find directory

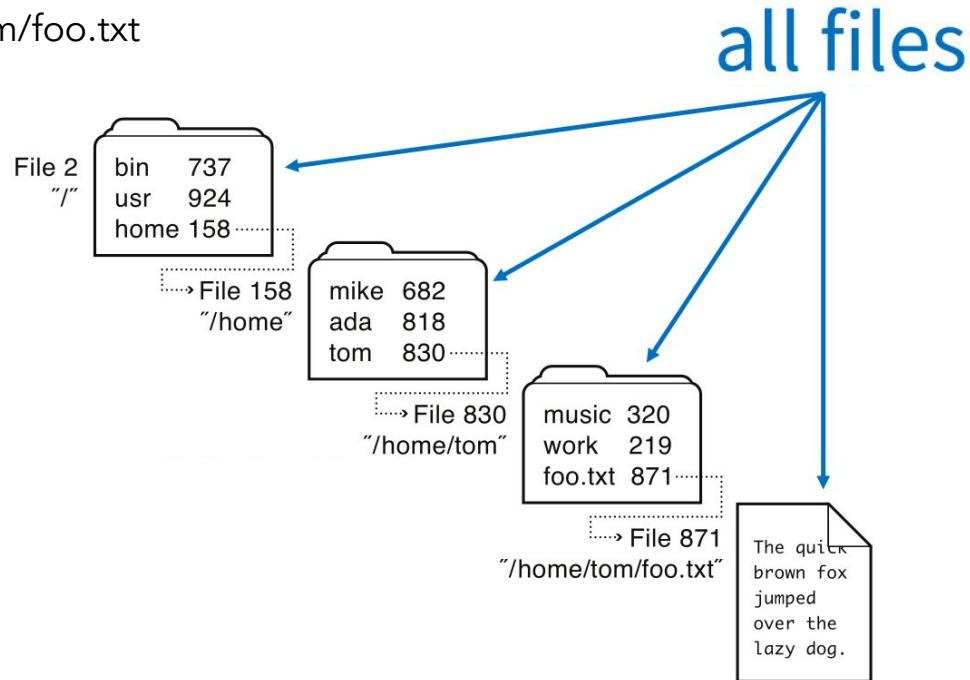
➤ Example

- /home/tom/foo.txt

OS uses path name to find directory

➤ Example

- /home/tom/foo.txt



Basic file system operations

Basic file system operations

➤ 6 operations

Basic file system operations

➤ 6 operations

- Create a file
- Write to a file
- Read from a file
- Seek to somewhere in a file
- Delete a file
- Truncate a file

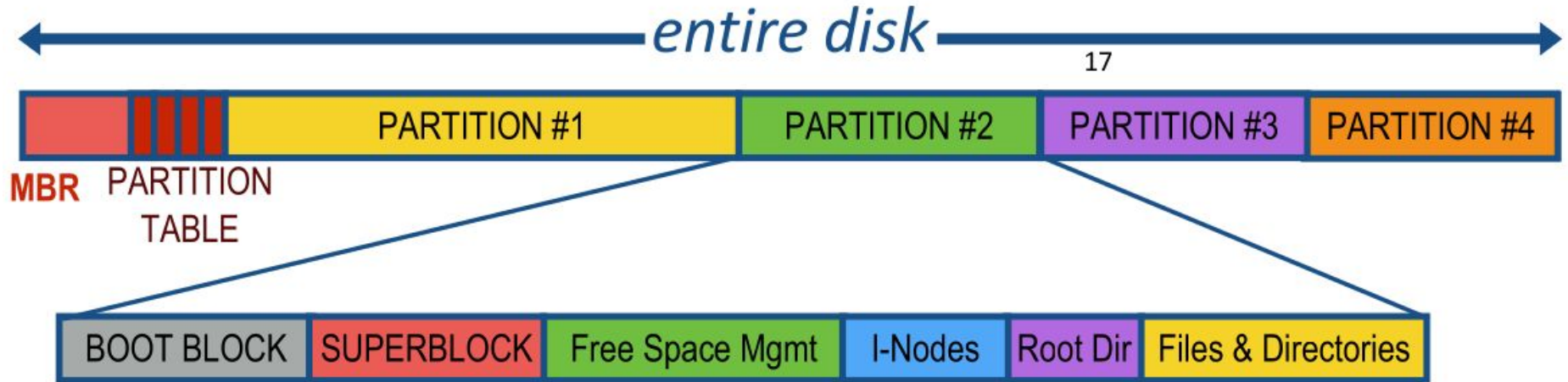
Challenges for file system designers

Challenges for file system designers

- Performance: despite limitations of disks
 - leverage spatial locality
- Flexibility: need jacks-of-all-trades, diverse workloads
 - not just FS for X
- Persistence: maintain/update user data + internal data
 - structures on persistent storage devices
- Reliability: must store data for long periods of time
 - despite OS crashes or HW malfunctions
- Security: file should have protection mechanisms

File system layout

File system layout



Characteristics of files

A Five-Year Study of File-System Metadata

NITIN AGRAWAL

University of Wisconsin, Madison

and

WILLIAM J. BOLOSKY, JOHN R. DOUCEUR, and JACOB R. LORCH

Microsoft Research

Published in FAST 2007

Characteristics of files

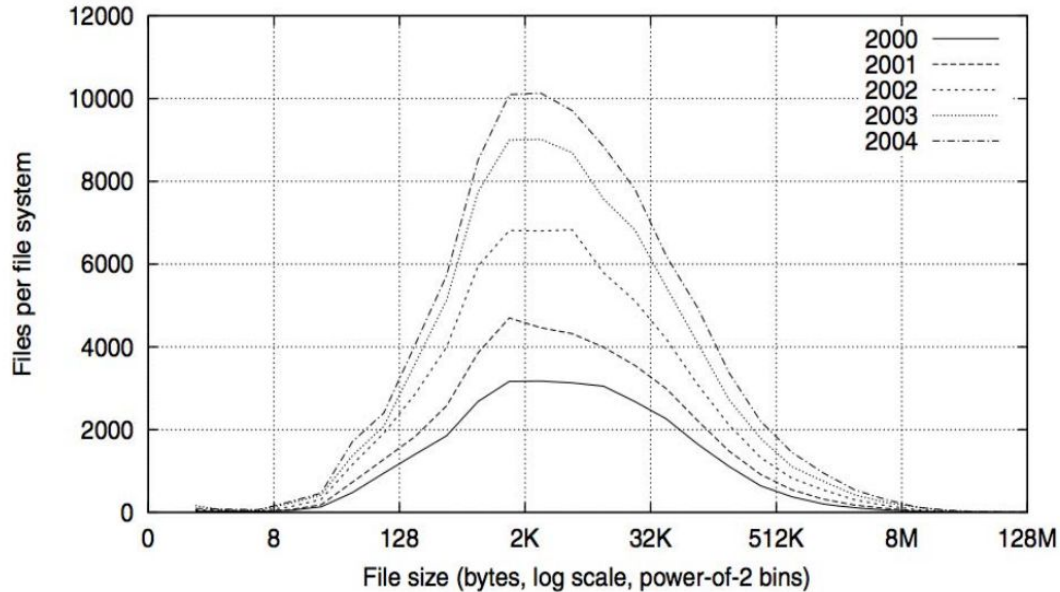


Fig. 2. Histograms of files by size.

Most files are small

Characteristics of files

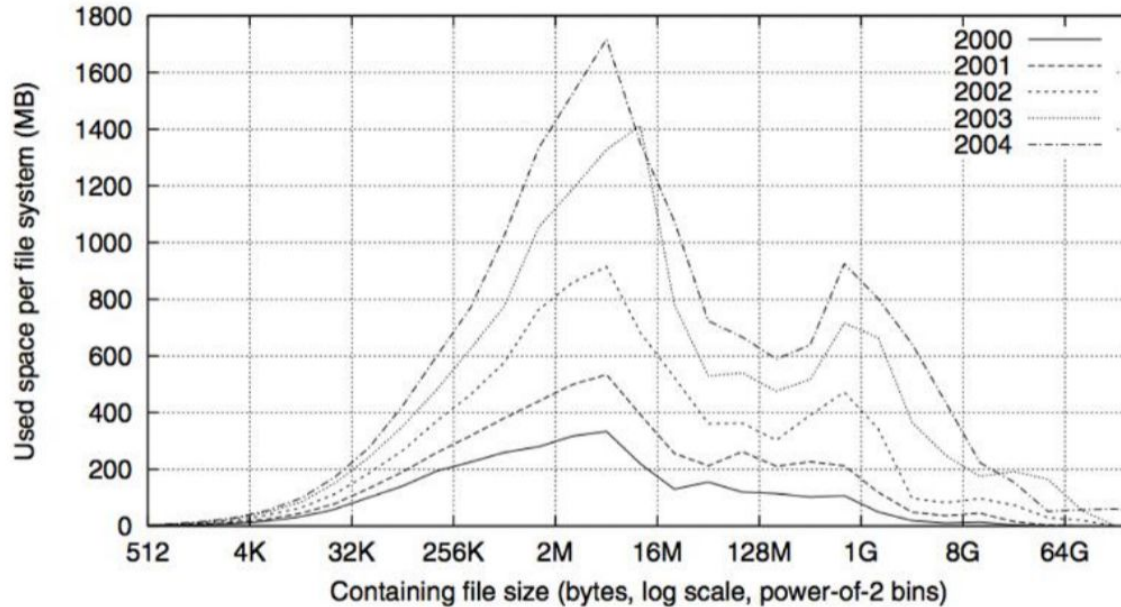


Fig. 4. Histograms of bytes by containing file size.

Most bytes are in large files

Storing files

Storing files

- Simple ways

Storing files

- Simple ways
 - Contiguous allocation

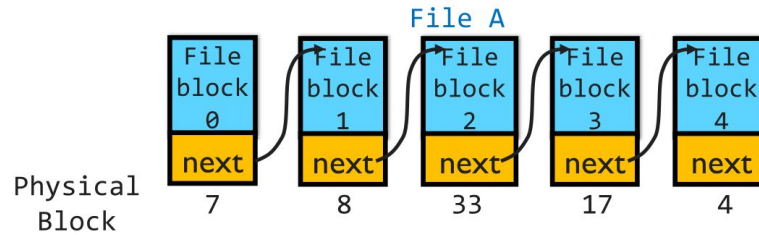


Storing files

- Simple ways
 - Contiguous allocation

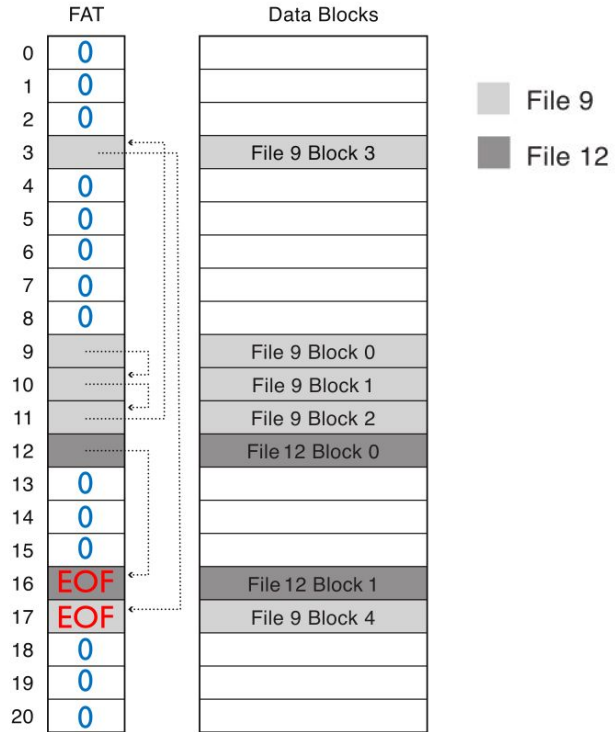


- Linked Structure

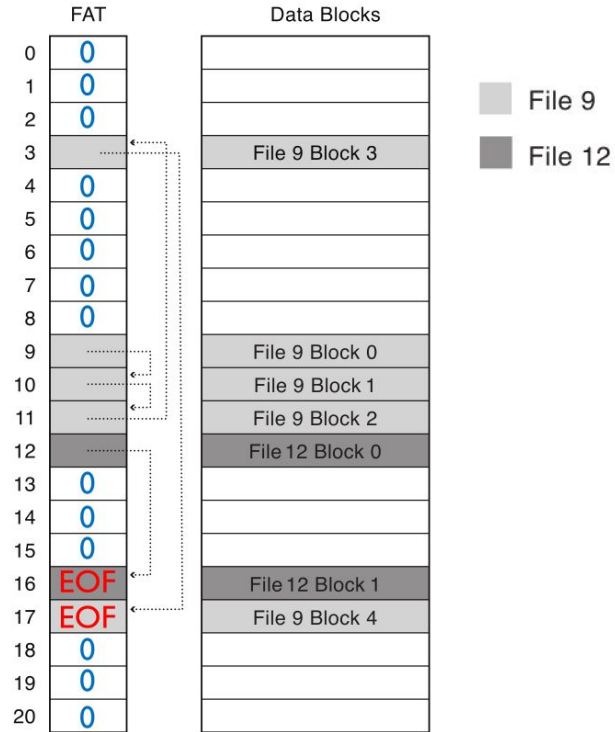


FAT (file allocation table)

FAT (file allocation table)

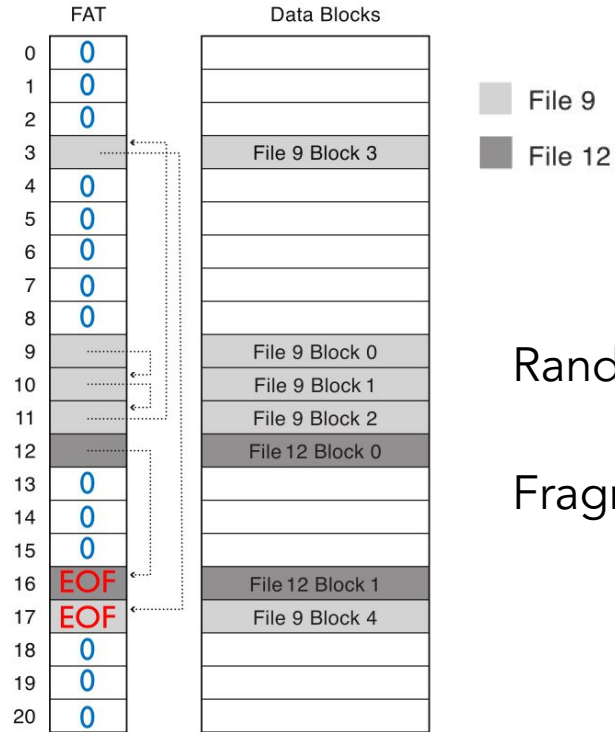


FAT (file allocation table)



Where is FAT stored?

FAT (file allocation table)

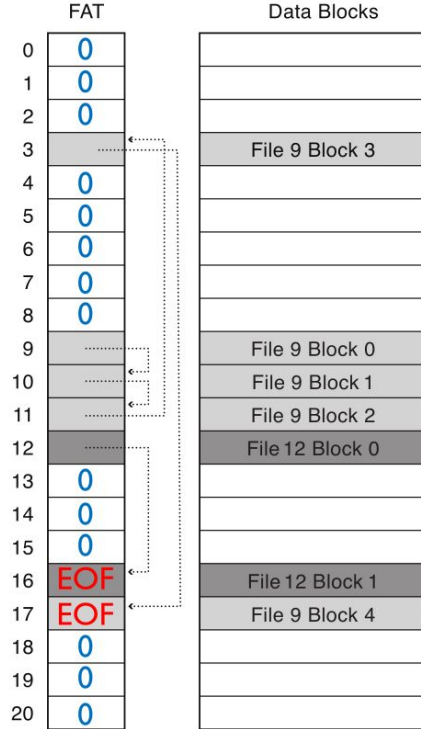


Random access?

Fragmentation?

Where is FAT stored?

FAT (file allocation table)



File 9

File 12

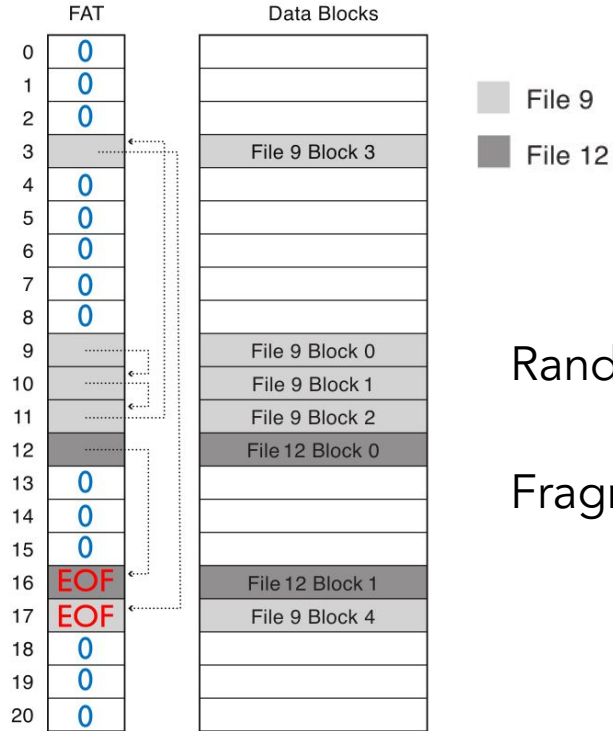
Directory	
bart.txt	9
maggie.txt	12

Random access?

Fragmentation?

Where is FAT stored?

FAT (file allocation table)



Where is FAT stored?

Directory	
bart.txt	9
maggie.txt	12

Random access?

Fragmentation?

Directory

- Essentially a file containing <file_name: file_number> mappings
- Each directory is a linked list of entries
- Where do you find root directory ("/")?

Example

- How many disk accesses to resolve `"/my/book/count"`?

Example

- How many disk accesses to resolve `"/my/book/count"`?
 - Read in file header for root (fixed spot on disk)
 - Read in first data block for root; search for `"my"`
 - Read in file header for `"my"`
 - Read in first data block for `"my"`; search for `"book"`
 - Read in file header for `"book"`
 - Read in first data block for `"book"`; search for `"count"`
 - Read in file header for `"count"`

Problem with FAT

Problem with FAT

- FAT has no access rights
 - No way, even in principle, to track ownership of data
- FAT has no header in the file blocks
 - No way to enforce control over data, since all processes have access of FAT table
 - Just follow pointer to disk blocks
- Just gives an index into the FAT to read data
 - (file number = block number)
 - Could start in middle of file or access deleted data

Unix file system (UFS)

Unix file system (UFS)

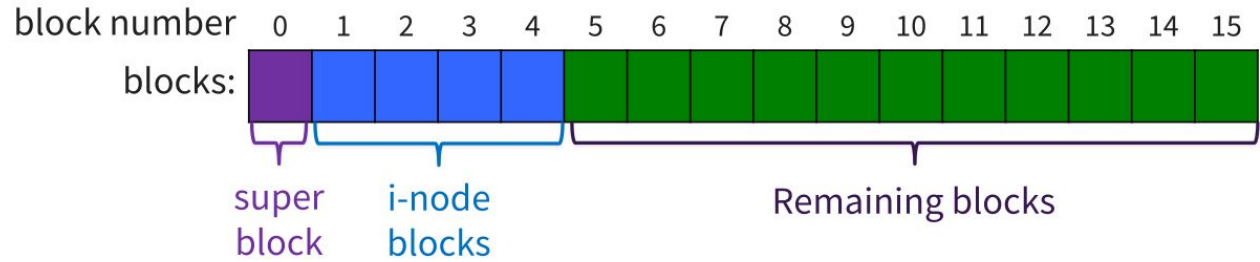
Tree-based

Multi-level index

Unix file system (UFS)

Tree-based

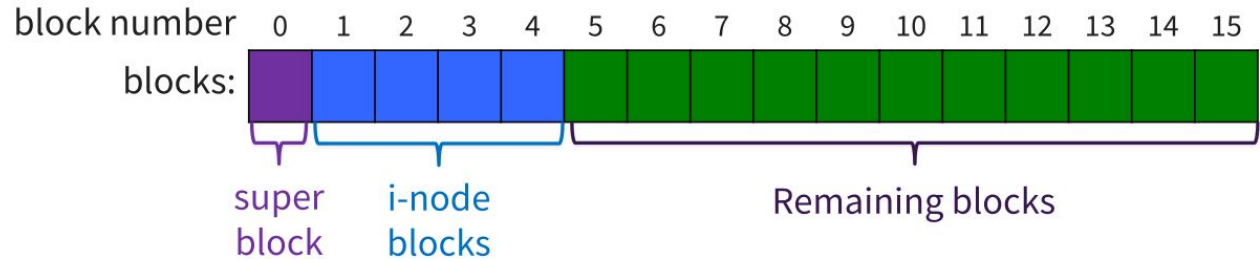
Multi-level index



Unix file system (UFS)

Tree-based

Multi-level index



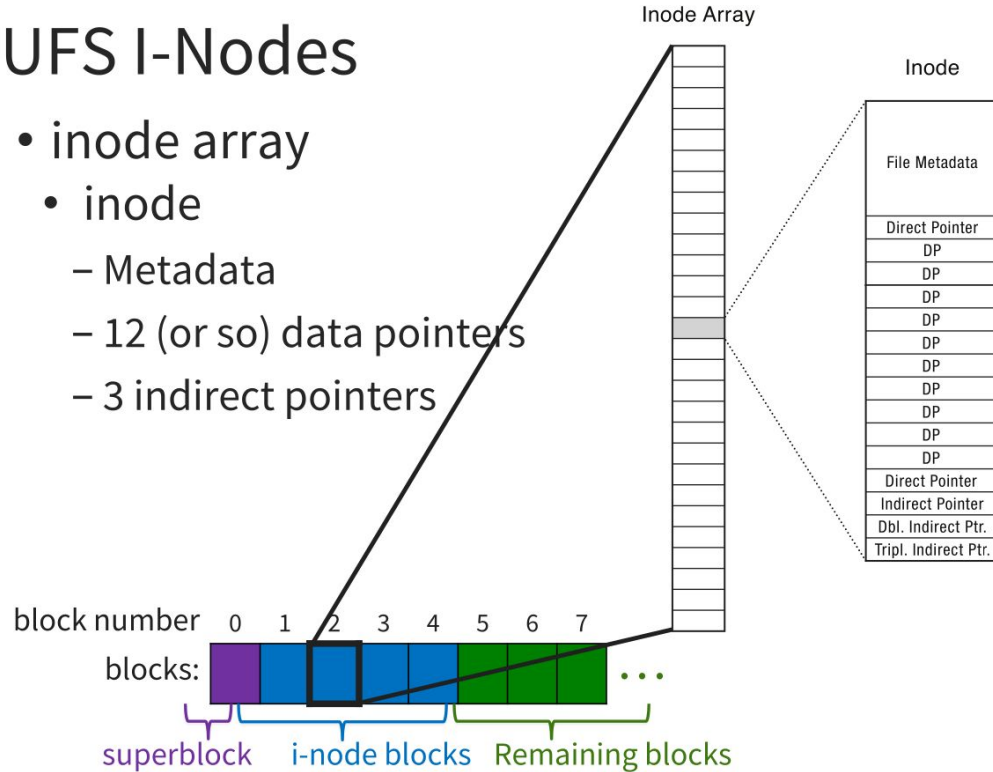
- Super block
 - type
 - block size
 - inode array location and size
 - location of free list

I-node

I-node

UFS I-Nodes

- inode array
- inode
 - Metadata
 - 12 (or so) data pointers
 - 3 indirect pointers

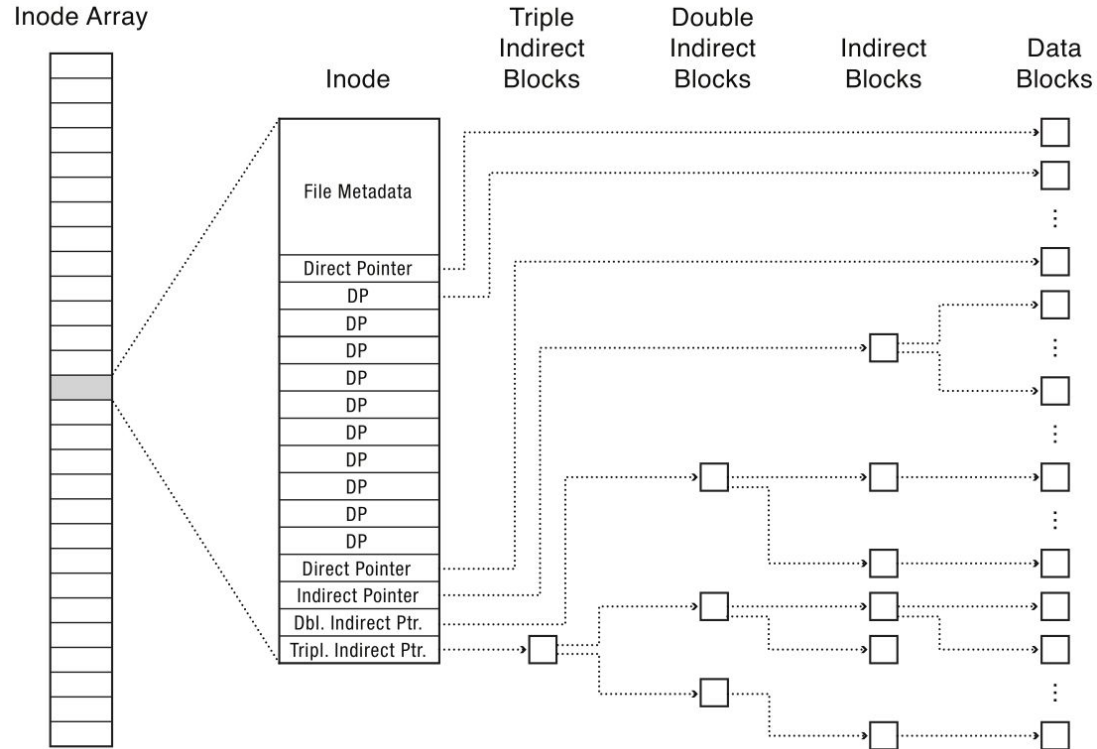


Metadata

- Type
 - ordinary file
 - directory
 - symbolic link
 - special device
- Size of the file (in #bytes)
- # links to the i-node
- Owner (user id and igroupd)
- Protection bits
- Times: creation, last accessed, last modified

File Metadata	
Direct Pointer	
DP	
DP	
DP	
DP	
DP	
DP	
DP	
DP	
DP	
Direct Pointer	
Indirect Pointer	
Dbl. Indirect Ptr.	
Tripl. Indirect Ptr.	

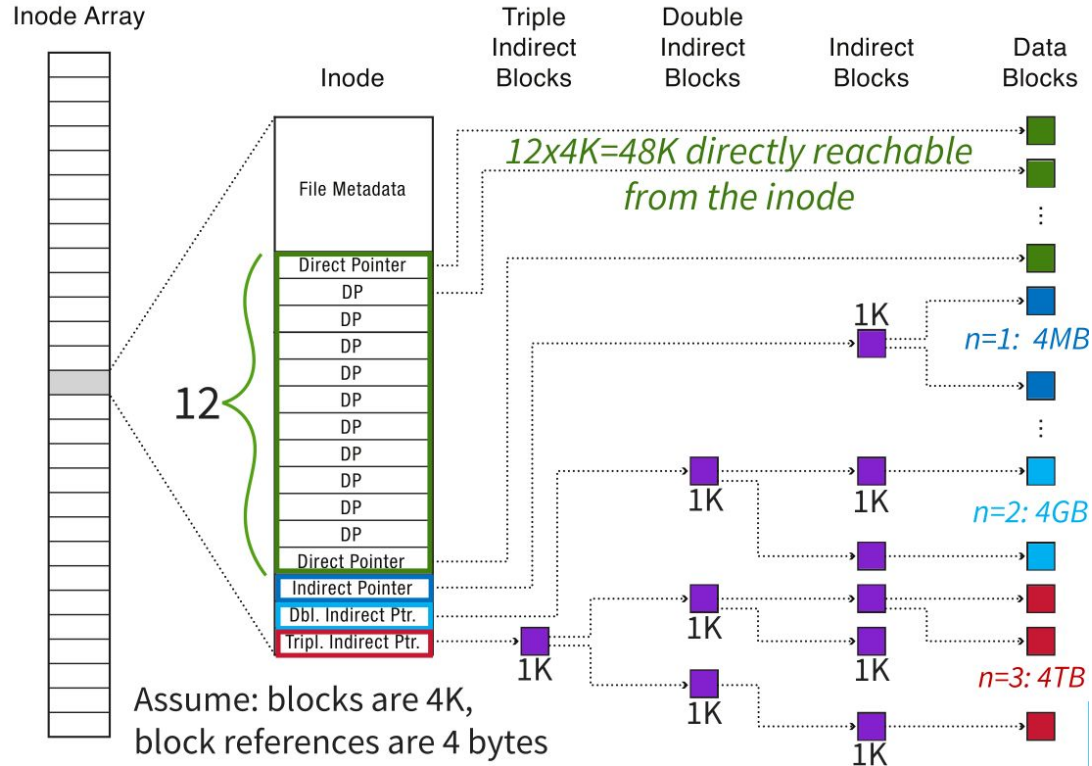
Index structures



Example

- 10 direct ptrs, 1K blocks
- How many accesses for block #23 (assume file header accessed on open)?
 - Two: One for indirect block, one for data
- How about block #5?
 - One: One for data
- Block #340?
 - Three: double indirect block, indirect block, and data

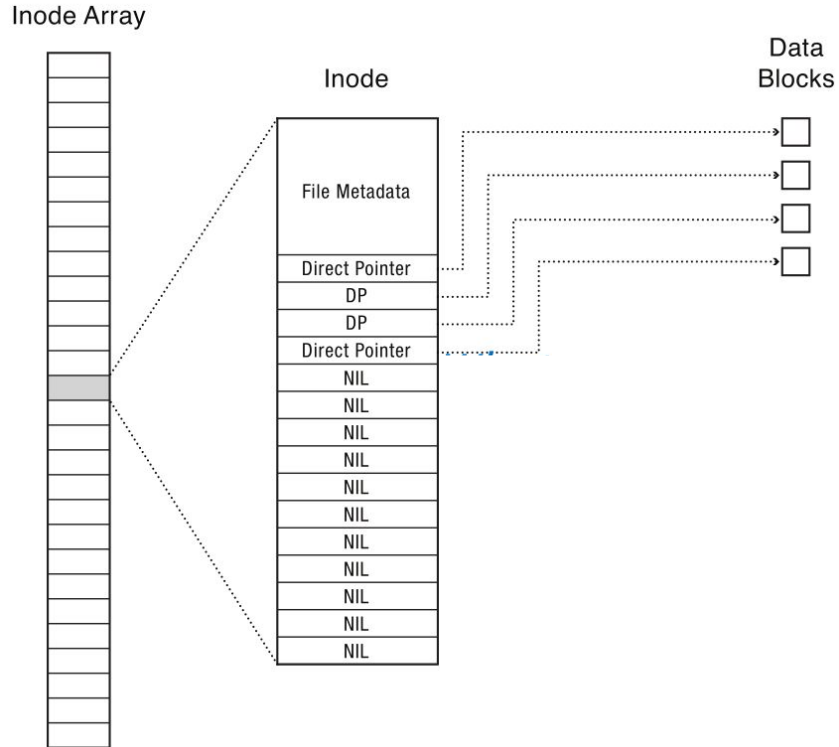
Fast file system (BSD 4.2)



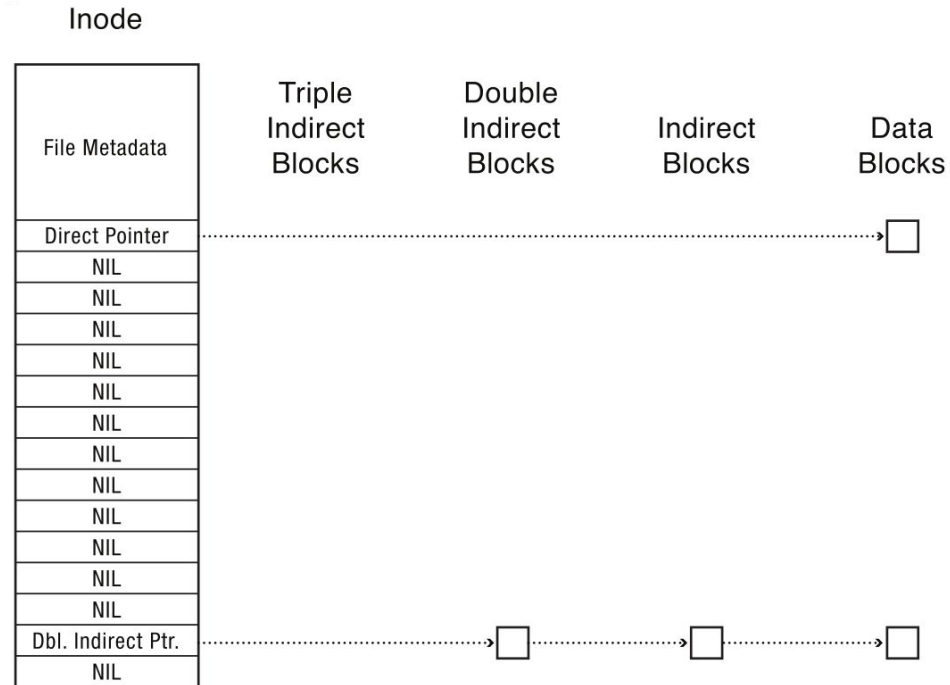
Characteristics of UFS

- 1. Tree structure
 - efficiently find any block of a file
- 2. High degree (or fan out)
 - minimizes number of seeks
 - supports sequential reads & writes
- 3. Fixed structure
 - implementation simplicity
- 4. Asymmetric
 - not all data blocks are at the same level
 - supports large files
 - small files don't pay large overheads

Small files in UFS



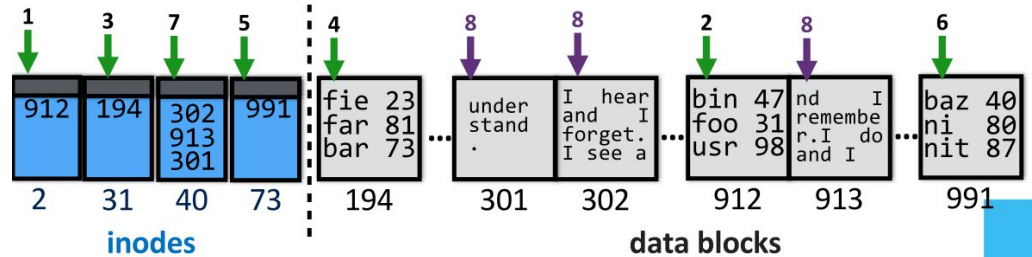
Sparse files in UFS



Reading /foo/bar/baz

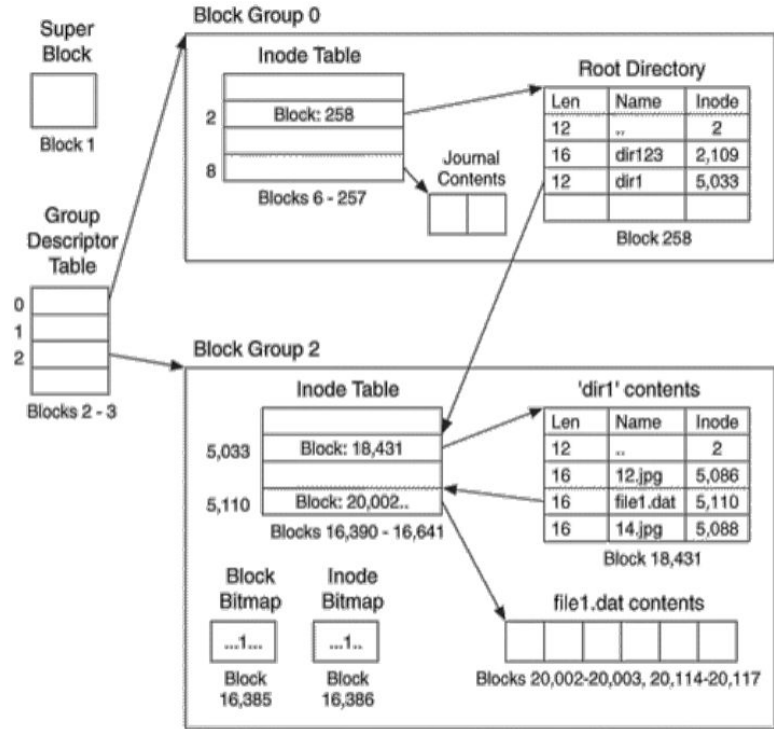
➤ Read & Open:

- inode #2 (root always has inumber 2), find root's blocknum (912)
- root directory (in block 912), find foo's inumber (31)
- inode #31, find foo's blocknum (194)
- foo (in block 194), find bar's inumber (73)
- inode #73, find bar's blocknum (991)
- Caching often allows
- bar (in block 991), find baz's inumber (40)
- inode #40, find data blocks (302, 913, 301) first few steps to be
- skipped
- data blocks (302, 913, 301)



Ext2/3 disk layout

- Disk divided into block groups
 - Each group has two block-sized bitmaps (free blocks/inodes)
 - Block sizes settable at format time:
 - 1K, 2K, 4K, 8K...
- Ext3: Ext2 with Journaling

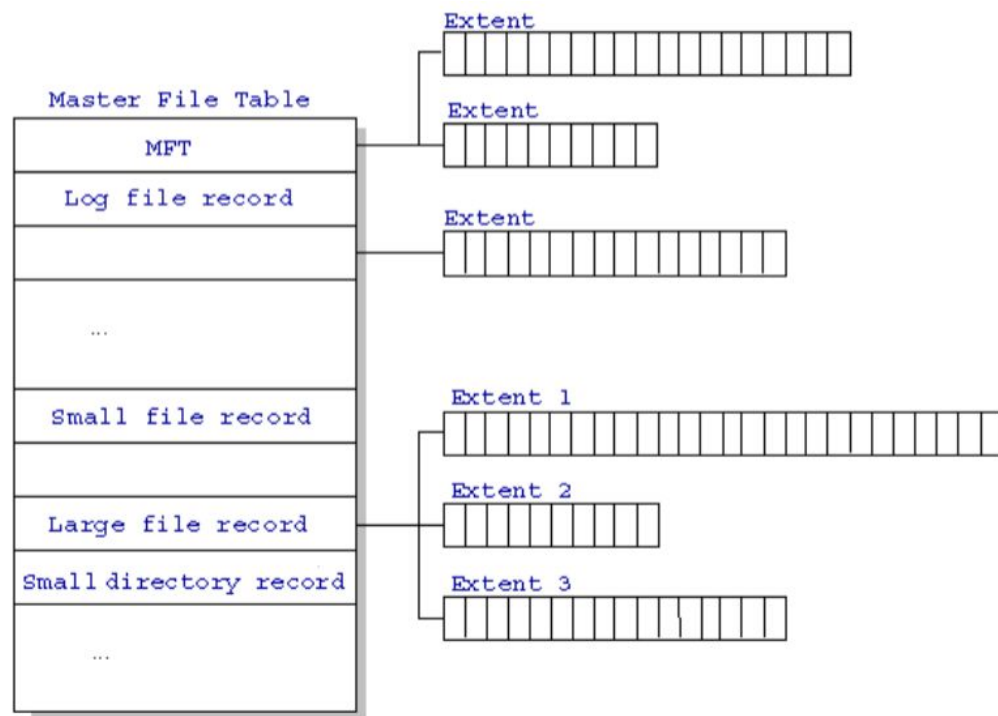


- Example: create a file1.dat under /dir1/ in Ext3

New technology file system (NTFS)

- Default on modern Windows systems
- Variable length extents
 - Rather than fixed blocks
- Instead of FAT or inode array: Master File Table
 - Like a database, with max 1 KB size for each table entry
 - Everything (almost) is a sequence of <attribute:value> pairs
- Meta-data and data
- Each entry in MFT contains metadata and:
 - File's data directly (for small files)
 - A list of extents (start block, size) for file's data
 - For big files: pointers to other MFT entries with more extent lists

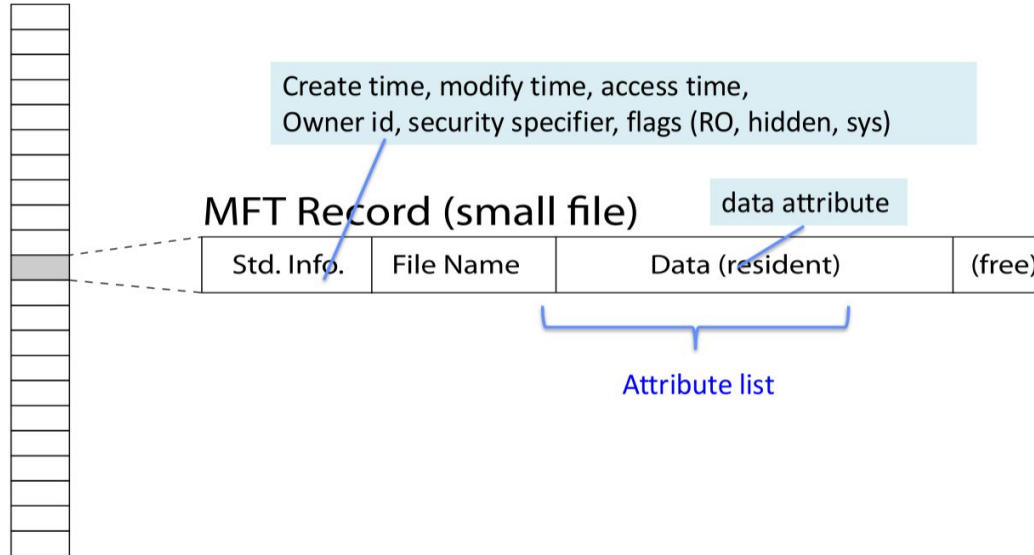
NTFS



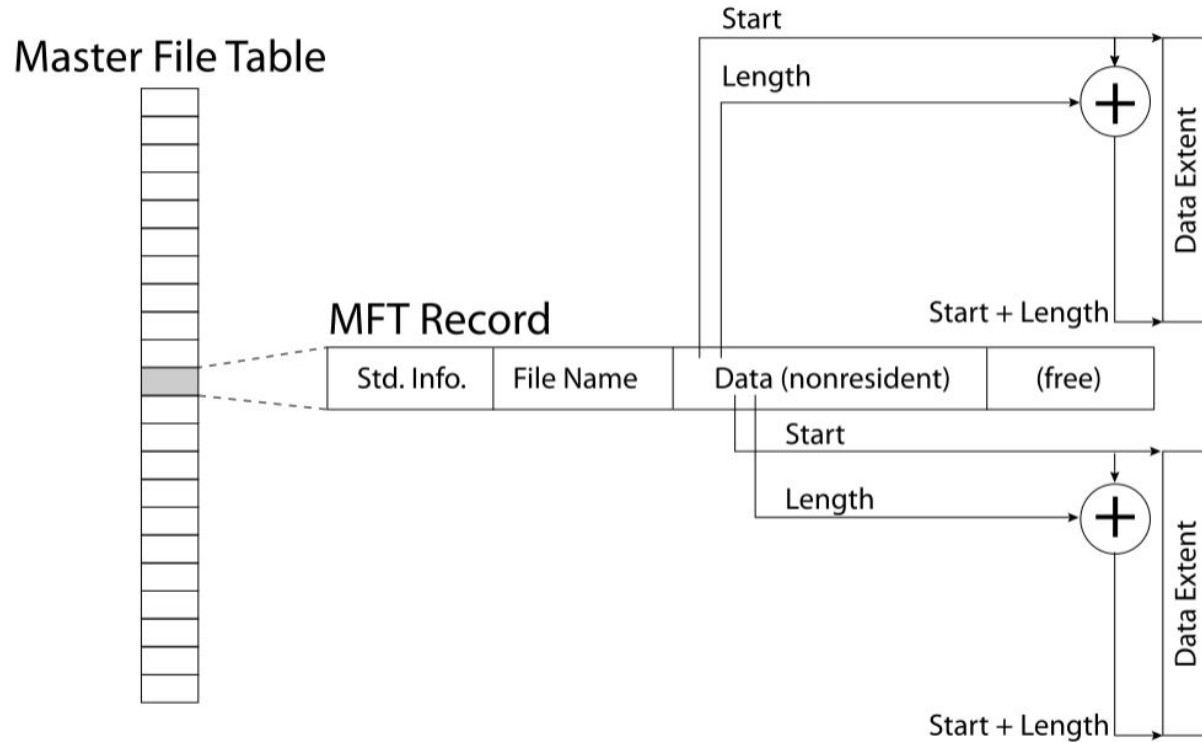
<http://ntfs.com/ntfs-mft.htm>

Small files

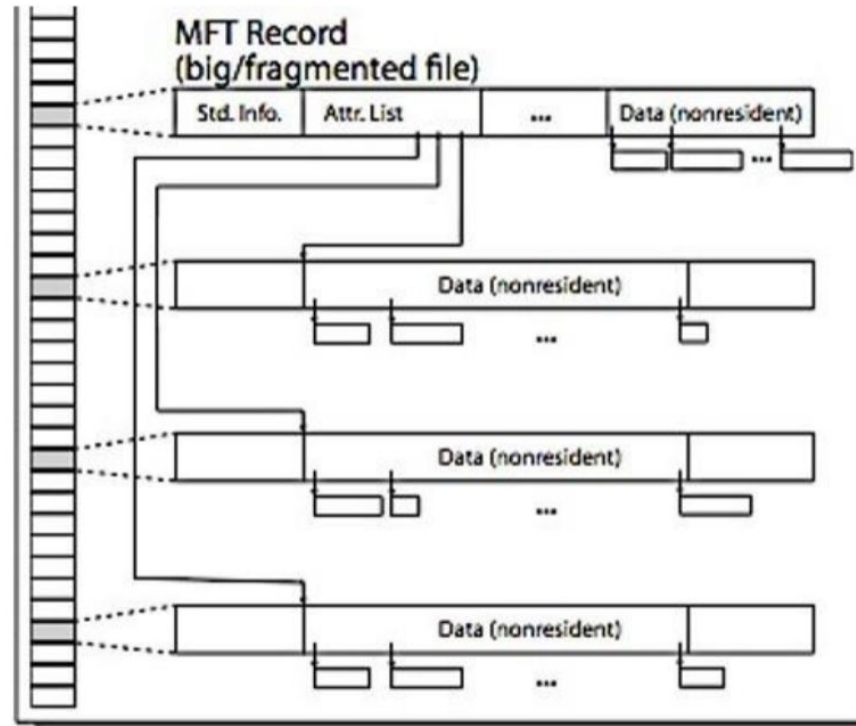
Master File Table



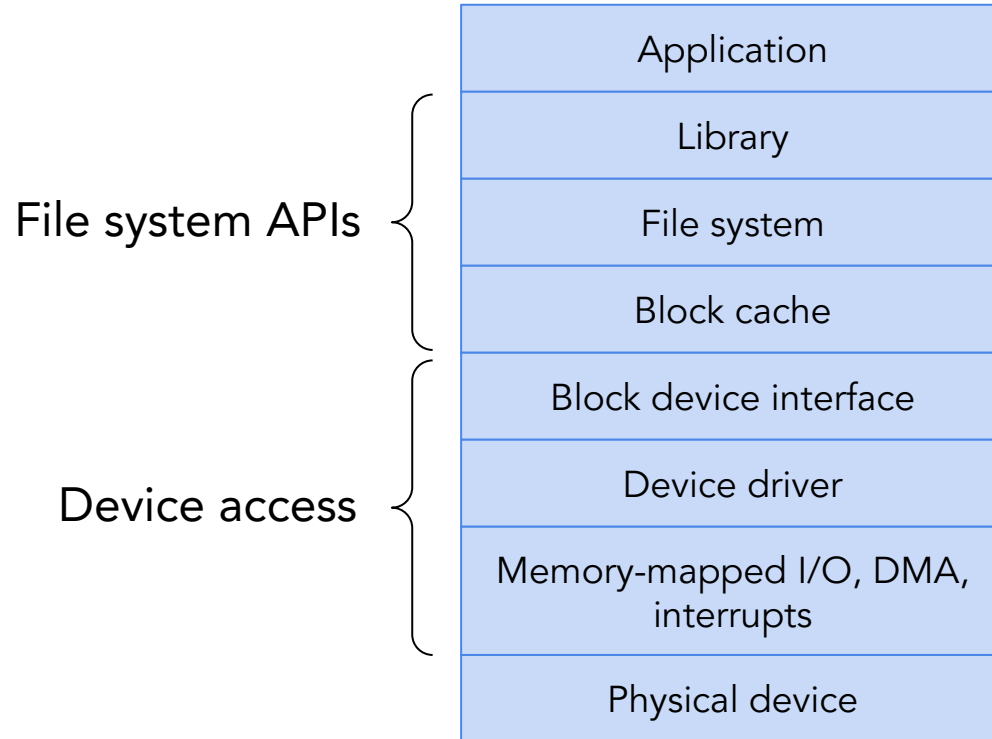
Medium files



Large files



Cache between file system and devices



Summary

➤ File System

- Transforms blocks into files and directories
- Optimize for access and usage patterns
 - Many small files, but large files take up all the space
- Maximize sequential access, allow efficient random access

➤ File allocation table (FAT) scheme

- Linked-list approach
- Very widely used: Cameras, USB drives, SD cards
- Simple to implement, but poor performance and no security

➤ 4.2 BSD fast file system: Multi-level inode header to describe files

- Inode contains pointers to actual blocks, indirect blocks, double indirect blocks, etc.

➤ NTFS: variable extents not fixed blocks, tiny files data is in header